

laSallefy

Reproductor musical con sintetizador propio

laSalle

UNIVERSIDAD RAMON LLULL

Asignatura: Programación Orientada a Objetos

Curso: PEI 2025-2026

Centro: La Salle Campus Barcelona

Autores: Jan Pérez y Xavi Bosch

Fecha de entrega: 28 de Mayo 2026

Memoria del proyecto final

Índice

1. Resumen del enunciado
2. Diagrama de clases UML
3. Explicación del diseño
4. Opcionales y extras
5. Pruebas realizadas
6. Dedicación en horas
7. Uso de Inteligencia Artificial
8. Conclusiones
9. Bibliografía

1. Resumen del enunciado

El proyecto consiste en desarrollar una aplicación de consola en Java que se comporte como un reproductor musical reducido, inspirado en plataformas como Spotify pero a escala muy pequeña. La diferencia principal con un reproductor convencional es que la aplicación no trabaja con archivos de audio reales (MP3, WAV...) sino que genera el sonido en tiempo real mediante un sintetizador propio construido sobre la API `javax.sound.sampled`.

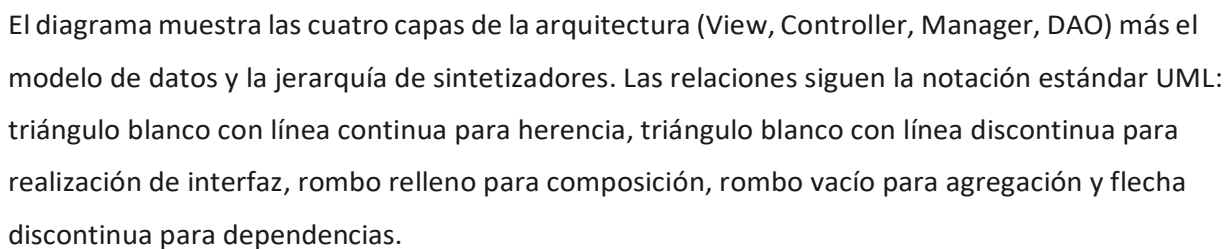
El sistema debe permitir gestionar una biblioteca de canciones (crearlas, listarlas, eliminarlas), agruparlas en playlists, reproducirlas y generar álbumes aleatorios filtrando por mood. Cada canción está definida por una serie de propiedades (título, artista, duración, mood, style) y opcionalmente por una secuencia de sonidos que la convierten en reproducible. Cada sonido es una tripleta de frecuencia, duración y timbre.

Toda la información debe persistirse en ficheros JSON usando la librería Gson, vista en la sesión 14 del temario. El diseño tiene que respetar la arquitectura por capas (View, Controller, Manager, DAO) y aplicar los principios de la programación orientada a objetos y los patrones GRASP.

Funcionalidades exigidas

- Gestión de canciones.
- Gestión de playlists.
- Añadir y quitar canciones de una playlist.
- Reproducir canciones individualmente y playlists completas.
- Generar un álbum aleatorio en base a un mood y una duración objetivo.
- Persistir todo en JSON usando Gson.
- Cuatro timbres distintos: sinusoidal, cuadrado, triangular y diente de sierra.

El siguiente diagrama representa la estructura completa del proyecto agrupada por capas, siguiendo las convenciones UML estándar.



3. Explicación del diseño

Arquitectura por capas

Hemos seguido la arquitectura por capas que se enseña en el temario porque es la que mejor encaja con la dimensión y complejidad del proyecto. La idea de separar responsabilidades en cuatro capas bien definidas nos ha ayudado a tener el código ordenado y a saber siempre dónde poner cada cosa nueva.

Hemos materializado esta separación en una estructura de paquetes que refleja directamente las capas: model, dao, manager, controller y view. Hemos añadido un paquete extra synth donde vive la clase abstracta SoundSynth y sus cuatro subclases (Sine, Square, Triangle, Sawtooth). De esta forma cualquier persona que abra el proyecto entiende a simple vista la arquitectura sólo mirando la estructura de carpetas.

El flujo siempre va del usuario hacia el almacenamiento siguiendo el orden: View → Controller → Manager → DAO → JSON. Cada capa solo conoce la capa inmediatamente inferior, lo que mantiene el acoplamiento bajo. Por ejemplo, el Controller nunca toca el DAO directamente, sino que pide al Manager que se encargue de la persistencia.

La capa View se encarga únicamente de la interacción con el usuario. La hemos modelado como interfaz para poder cambiar la implementación sin tocar el resto del programa. Todas las interacciones con System.in y System.out están concentradas aquí, no aparecen en DAOs ni en modelos.

El Controller se limita a orquestrar. Recibe la entrada del usuario, decide qué Manager debe ejecutar la lógica y muestra el resultado. No contiene reglas de negocio.

Los Managers (LibraryManager, PlaybackManager y AlbumGenerator) contienen toda la lógica del dominio. Saben gestionar la biblioteca, comprobar duplicados, validar borrados, coordinar la reproducción, generar álbumes aleatorios, etc.

La capa DAO abstrae el acceso a la persistencia.

Modelo de datos

Hemos seguido el ejemplo de la sesión 16 del temario para definir las clases del modelo. La clase Sound es concreta y representa un único sonido con tres atributos: frequency (int, en Hz), durationMs (int, en milisegundos) y timbre (String con valores SINE, SQUARE, TRIANGLE o SAWTOOTH). Una canción es una lista ordenada de objetos Sound.

La clase Song contiene id (String, formato s001, s002...), title, artist, durationSeconds, mood, style, un campo playable explícito y la lista de sonidos. El método isPlayable() combina el campo

playable con la comprobación de que la lista de sonidos no esté vacía, así garantizamos que solo se puedan reproducir canciones que realmente tengan datos.

Cuando el usuario añade una canción nueva desde el menú, le preguntamos si quiere que sea reproducible. Si dice que sí, escoge un timbre y una de las cuatro plantillas de melodía que tenemos predefinidas en código (Happy Birthday, una escala, un arpeggio y un tono de aviso). Cada plantilla es simplemente una lista de objetos Sound con sus frecuencias y duraciones. De esta forma cumplimos lo que pide el enunciado (poder crear canciones reproducibles eligiendo una melodía predefinida) sin obligar al usuario a teclear nota a nota.

La clase Playlist tiene id (formato p001, p002...), name, description, mood objetivo, una lista de songIds (Strings, no objetos canción, solo referencias) y un campo random que indica si la playlist fue generada automáticamente por el AlbumGenerator.

Capa de persistencia con DAO

La integración con Gson ha sido directa gracias a que Sound es una clase concreta. El DAO de canciones simplemente hace `gson.fromJson(reader, Song[].class)` y obtiene el array completo deserializado de un solo tirón. Para guardar, llamamos a `gson.toJson(songs)` y escribimos el resultado en el fichero con un `FileWriter`. Hemos activado `setPrettyPrinting()` para que los JSONs queden legibles.

Sintetizadores de sonido

La jerarquía de sintetizadores sigue exactamente el patrón de la sesión 13 del temario. `SoundSynth` es una clase abstracta que declara un único método abstracto: `makeSound(int frequency, int durationMs)`. Las cuatro subclases (`SoundSynthSine`, `SoundSynthSquare`, `SoundSynthTriangle` y `SoundSynthSawtooth`) implementan cada una su versión completa del método, calculando la forma de onda correspondiente.

El `PlaybackManager` mantiene un `Map<String, SoundSynth>` que asocia cada timbre con su sintetizador. Cuando reproduce una canción, recorre la lista de sonidos y por cada uno escoge el sintetizador adecuado consultando el campo timbre del Sound.

Patrones GRASP aplicados

- **Information Expert:** cada clase contiene la lógica que opera sobre los datos que tiene. Por ejemplo, `Playlist.getTotalDurationSeconds` recibe el mapa de canciones y calcula la duración total, porque es ella quien tiene la lista de IDs.
- **Creator:** el `LibraryManager` es quien crea las playlists generadas automáticamente (a través del `AlbumGenerator`) y persiste las canciones nuevas creadas desde el `Controller`.
- **Controller:** tenemos un único `Controller` que recibe todas las peticiones del usuario, separando claramente la lógica de presentación de la de negocio.
- **Low Coupling:** uso de interfaces para la capa DAO y para la View. Las clases concretas pueden cambiar sin romper el resto.

- **High Cohesion:** cada clase tiene una responsabilidad clara. El DAO solo persiste, el Manager solo opera, el Controller solo coordina.
- **Polymorphism:** el ejemplo más claro es la jerarquía SoundSynth con sus cuatro subclases.

Problemas encontrados

Al principio habíamos pensado tener dos tipos de sonido distintos (notas y silencios) como clases separadas que heredaban de una clase Sound abstracta. El problema apareció al guardar y cargar los JSON con Gson, como la librería no sabía a cuál de los dos tipos pertenecía cada elemento, teníamos que ir construyendo cada sonido a mano. Después de mirar el material de la sesión 16 decidimos hacerlo más sencillo: una sola clase Sound con tres campos (frecuencia, duración y timbre). El código del DAO quedó mucho más corto y Gson hace todo el trabajo por nosotros.

El otro problema vino con la reproducción de canciones. Al principio todo se ejecutaba en el mismo hilo que el menú, así que mientras sonaba una canción no podíamos hacer nada. Cuando movimos la reproducción a un hilo aparte, tuvimos que aprender a usar la palabra clave volatile en las variables que se comparten entre los dos hilos. Es una palabra que no habíamos usado nunca y nos costó un poco entender por qué hace falta, pero al final vimos que sin ella los cambios que hace un hilo no los ve el otro hasta más tarde, y eso provocaba que la pausa no funcionara bien.

Cosas que mejoraría con más tiempo

- Permitir introducir las notas de una canción una a una desde el menú, en lugar de elegir solo entre plantillas predefinidas.

Hacer una versión gráfica simple.

4. Opcionales y extras

Más allá de lo estrictamente exigido en el enunciado, hemos añadido algunos elementos que enriquecen el proyecto.

Reproducción en hilo aparte

Hemos implementado la reproducción de canciones en un Thread separado para que la interfaz no quede bloqueada mientras suena una canción. Para coordinar el hilo principal con el hilo de reproducción usamos dos flags volatile boolean (paused y stopped) que el bucle de reproducción consulta entre sonido y sonido, ayudándonos de la teoría de la sesión 17 del GitHub.

Menú de control durante reproducción

Como complemento al thread, hemos añadido un pequeño menú interactivo que aparece mientras la canción está sonando. La opción es **P** para pausar y reanudar, y **ENTER** para parar la reproducción y volver al menú. Este menú demuestra el valor real del hilo separado, ya que sin él no podríamos interactuar con el programa hasta que acabara la canción.

README detallado

Hemos escrito un README.md detallado en el proyecto con instrucciones de compilación, ejemplos de uso y una explicación de la arquitectura. El enunciado lo marca como opcional pero recomendable.

5. Pruebas realizadas

Hemos validado el correcto funcionamiento del programa siguiendo un plan de pruebas manual que cubre todas las funcionalidades.

#	Funcionalidad	Verificación
1	Listar canciones	Aparecen las 10 canciones del JSON con su etiqueta PLAYABLE o NOT PLAYABLE.
2	Añadir canción (no reproducible)	Tras introducir los datos y responder "no" a reproducible, la canción aparece con un id único correlativo (s011...) y etiqueta NOT PLAYABLE.
2b	Añadir canción reproducible	Al responder "sí", elegimos timbre y plantilla; la canción se crea con sus sonidos y aparece como PLAYABLE. Se puede reproducir y suena.
3	Eliminar canción no usada	La canción desaparece y el JSON se actualiza.
4	Bloqueo de borrado de canción usada	Al intentar borrar una canción referenciada en una playlist, el sistema lo rechaza con un mensaje claro.
5	Listar playlists	Aparecen las 4 playlists con número de canciones, duración total, reproducibles, no reproducibles y marca RANDOM si toca.
6	Crear playlist con y sin mood	La playlist se persiste correctamente con o sin mood asignado.
7	Añadir y quitar canciones de playlist	Los cambios persisten al volver a arrancar el programa.
8	Eliminar playlist	La playlist desaparece y las canciones siguen existiendo.
9	Reproducir canción	Se escucha la canción con el timbre correcto, y al terminar el control vuelve al menú.
10	Pausar y reanudar	Al teclear P la canción se pausa entre sonidos, y al teclear P de nuevo continúa.
11	Volver al menú	Al pulsar ENTER la reproducción se detiene y volvemos al menú.
12	Reproducir playlist completa	Suenan secuencialmente todas las canciones reproducibles de la playlist.
13	Tecla no válida al reproducir	Si tecleamos algo que no sea P ni ENTER, muestra "Opción no válida" y la música sigue sonando.

14	Generar álbum random	Se crea una playlist nueva con canciones del mood pedido, marcada como random=true, cerca del tiempo objetivo. Si pulsamos B al elegir el mood, se cancela y volvemos al menú sin crear nada.
15	Validar mood y style fuera de rango	Si se introduce un número incorrecto, muestra "Opción no válida" y vuelve a preguntar, sin romper el programa.
16	Validar IDs inexistentes	Reproducir o eliminar un id que no existe muestra un mensaje claro.
17	Persistencia tras reinicio	Al cerrar y volver a arrancar el programa, todos los cambios siguen ahí.
18	Opción no válida en menú	Cualquier letra o número no contemplado muestra "Opción no válida" y vuelve al menú.
19	Salir limpiamente	Q cierra el programa con el mensaje de despedida.

Adicionalmente hemos probado los cuatro timbres reproduciendo las canciones del JSON que usan cada uno de ellos (Happy Birthday usa SINE, Chiptune Rush usa SQUARE, etc.) y hemos confirmado que cada onda suena diferente.

6. Dedicación en horas

A continuación detallamos el tiempo aproximado invertido en cada fase del proyecto.

Fase	Horas	Detalle
Análisis del enunciado	2 h	Lectura del enunciado, repaso del temario relacionado.
Diseño UML inicial	3 h	Bocetos del diagrama de clases, identificación de relaciones.
Configuración del proyecto	1 h	Estructura de carpetas, integración de Gson, configuración del módulo IntelliJ.
Implementación del modelo	3 h	Clases Sound, Song, Playlist siguiendo el ejemplo de la sesión 16.
Implementación de los sintetizadores	5 h	SoundSynth abstracto y las cuatro subclases siguiendo la sesión 13.
Capa DAO con Gson	3 h	Interfaces SongsDAO/PlaylistsDAO con getAll/save y sus implementaciones JSON.
LibraryManager y AlbumGenerator	3 h	Lógica de negocio para biblioteca y generación de álbumes.
PlaybackManager y Controller	5 h	Coordinación de la reproducción y construcción de los menús.
Reproducción en hilo separado	3 h	Diseño de la concurrencia con flags volatile, menú de control durante reproducción.
Pruebas y depuración	4 h	Plan de pruebas manual, corrección de bugs encontrados.
Redacción de la memoria y README	4 h	Diagramas, explicaciones, plan de pruebas documentado.
TOTAL	36 h	Distribuidas en aproximadamente cuatro semanas.

7. Uso de Inteligencia Artificial

Durante el desarrollo del proyecto hemos utilizado herramientas de IA (Claude de Anthropic) como apoyo de estudio y consulta. En todo momento la implementación, las decisiones de diseño y la redacción han sido nuestras; la IA ha actuado como un tutor al que podíamos preguntar dudas conceptuales sin que nos resolviera el ejercicio.

A continuación detallamos los tipos de consultas que hemos realizado, agrupadas por temática.

Dudas conceptuales de POO

"Explícame con un ejemplo sencillo la diferencia entre una clase abstracta y una interfaz en Java."

"¿Cuándo es mejor usar composición y cuándo agregación al modelar una relación entre dosb clases?"

"¿Para qué sirven los getters y setters si los atributos ya son privados?" • "¿Qué quiere decir que un método sea @Override?"

Sintaxis de Java

"¿Qué diferencia hay entre ==y .equals()en Strings?"

"¿Cuándo debo usar static?"

Persistencia con Gson

"¿Cómo funciona gson.fromJson(reader, Clase[].class)?"

"¿Qué diferencia hay entre JsonReadery FileReaderal leer un JSON con Gson?"

Excepciones

"¿Qué diferencia hay entre una excepción checked y una unchecked?"

"¿Cuándo conviene crear una excepción propia como DataAccessException?"

Sobre el proyecto

"¿Es buena idea que el ID de una canción sea un String en lugar de un int?"

"¿Cómo organizo los paquetes en un proyecto Java?"

En ningún momento hemos pedido a la IA que generase clases enteras, métodos complejos o el diagrama UML por nosotros. La IA nos ha servido para entender mejor los conceptos teóricos y para verificar nuestras propias decisiones de diseño, no como sustituto del trabajo personal.

8. Conclusiones

A nivel personal

Este proyecto ha sido el primero en el que hemos podido aplicar de verdad todo lo que llevamos visto en el curso. Aquí hemos tenido que combinar herencia, polimorfismo, interfaces, persistencia y arquitectura por capas en un solo proyecto coherente. Al final del proceso nos hemos dado cuenta de que entendemos mucho mejor por qué cada uno de estos conceptos existe y para qué sirve.

Una de las cosas más satisfactorias ha sido escuchar por primera vez una canción saliendo del programa después de haber implementado los sintetizadores. Pasar de unas líneas de código a un sonido real es un buen recordatorio de que la programación puede tener resultados muy tangibles.

También nos hemos enfrentado por primera vez a problemas de concurrencia al añadir el hilo de reproducción. Gracias a este proyecto hemos podido entender lo importante que es todo el diseño, organización y planteamiento previo a empezar a picar código.

A nivel técnico

El proyecto demuestra que un buen diseño basado en capas y patrones GRASP simplifica mucho el desarrollo, especialmente cuando aparecen requisitos nuevos. Por ejemplo, añadir el thread de reproducción solo nos obligó a modificar el `PlaybackManager` y el `Controller`, sin tocar el modelo de datos ni la persistencia.

El uso de interfaces para los DAO y para la View ha sido una decisión muy acertada ya que nos permite cambiar la implementación sin tocar nada más, y nos ha enseñado de primera mano qué quiere decir bajo acoplamiento.

Hemos aprendido que un diseño más simple suele ser mejor que uno más complejo. Al principio queríamos tener notas y silencios como dos clases separadas, pero el modelo más sencillo de la sesión 16, con una única clase `Sound`, funciona igual de bien y nos ha ahorrado mucha complicación al guardar y cargar los JSON.

En definitiva, creemos que el proyecto cumple sobradamente con lo que pide el enunciado y que las decisiones de diseño tomadas hacen que sea fácil de mantener y de extender. Si en el futuro tuviéramos que añadir nuevos tipos de sonidos, nuevas formas de persistencia o una interfaz gráfica, sabemos exactamente dónde y cómo hacerlo.

9. Bibliografía

Referencias siguiendo el estilo APA 6th edition.

1. Google. Gson User Guide — serialización y deserialización JSON en Java.
<https://github.com/google/gson/blob/main/UserGuide.md>
2. Baeldung. Java: programación orientada a objetos, colecciones y excepciones.
<https://www.baeldung.com/java-tutorialc>
3. Visual Paradigm. UML Class Diagram Tutorial.
<https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>
4. La Salle Campus Barcelona. (2025). Repositorio del temario PEI 2025-2026.
<https://github.com/yurigo/PEI-2526>